Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE1113 Sistemas Empotrados
Prof. Dr.-Ing. Jeferson González Gómez

Tutorial: Yocto Project Parte II

1. Introducción

Este tutorial presenta la continuación a la herramienta del proyecto Yocto, para la generación de imágenes a la medida de Linux para sistemas embebidos. A continuación se describen los pasos para la generación de una imagen de Linux a la medida para la tarjeta Raspberry Pi 4, así como los pasos para la construcción de su cross-toolchain respectivo y un ejemplo básico de compilación cruzada sobre la tarjeta utilizando la imagen y el toolchain creado.

2. Pasos para construcción de imagen

1. Dentro del directorio poky-scarthgap-5.0/ descargar el paquete de soporte de tarjeta (BSP) para la plataforma Raspberry Pi 4:

   ```
   git clone -b scarthgap https://github.com/agherzan/meta-raspberrypi.git
   ```

2. Crear directorio para construcción de imagen:

   ```
   mkdir rpi4
   ```

3. Configurar el ambiente de Yocto para el directorio de construcción:

   ```
   source  oe-init-build-env rpi4
   ```

4. Editar archivo de configuración local para construcción (conf/local.conf). Agregar al final del archivo:

   ```
   MACHINE ?= "raspberrypi4"
   INHERIT += "rm_work"
   DL_DIR ?= "/home/usuario/poky-scarthgap-5.0/downloads"
   ```

   Esto corresponde respectivamente a: cambiar la máquina objetivo de construcción a Raspberry Pi 4, directiva para eliminar archivos intermedios en construcción, y cambiar directorio de descargas a utilizado anteriormente.

5. Agregar capa de soporte de tarjeta (BSP) para Raspberry Pi 4 (meta-raspberrypi) a archivo de configuración de capas de Yocto (conf/bblayers.conf). El archivo final debe verse como:

```
BBLAYERS ?= " \
   /home/usuario/poky-scarthgap-5.0/meta \
   /home/usuario/poky-scarthgap-5.0/meta-poky \
   /home/usuario/poky-scarthgap-5.0/meta-yocto-bsp \
   /home/usuario/poky-scarthgap-5.0/meta-raspberrypi \
  "
```

   Esto permite que se habilite la construcción de todas las recetas (.bb) y paquetes, en general, necesarios para la construcción de imágenes para la tarjeta Raspberry Pi 4.

6. Construir la imagen:

```
bitbake rpi-test-image
```

7. Al finalizar, podrá copiar la imagen a una tarjeta uSD de la forma tradicional:

   a) Ingresar a directorio de imagen creada:

```
cd tmp/deploy/images/raspberrypi4
```

   b) Copiar imagen a uSD (of=/dev/sdX o of=/dev/mmcblkX según corresponda). Yocto 5.0 puede generar archivos .wic.bz2 o .rpi-sdimg:

```
# Para archivo .rpi-sdimg (formato tradicional)
sudo dd if=rpi-test-image-raspberrypi4.rpi-sdimg of=/dev/mmcblk0 bs=1M
# Para archivo .wic (formato moderno)
sudo bmaptool copy rpi-test-image-raspberrypi4.rootfs.wic.bz2 /dev/sdX
sync
```

   Puede usar de igual manera el método con ddrescue. Al finalizar, la imagen está lista ser utilizada en la Raspberry Pi 4. El usuario para inicio de sesión es *root*

3. Pasos para construcción de cross-toolchain

En esta sección se describen los pasos para la construcción y uso del *toolchain* cruzado para desarrollo de aplicaciones para la imagen de Raspberry Pi 4, creada en la sección anterior.

1. Dentro de directorio raíz de yocto (poky-scarthgap-5.0). Ejecutar script para configuración de ambiente sobre el directorio de construcción de imagen para Raspberry Pi 4:

```
source oe-init-build-env rpi4
```

2. Crear toolchain. El paquete para el toolchain se construye de la misma manera que se construye una imagen para la plataforma, como sigue:

```
bitbake rpi-test-image -c populate_sdk
```

3. Una vez finalizado el proceso, debe instalar el toolchain. Se recomienda utilizar el directorio estándar sugerido por el script. Para esto debe ejecutarse:

```
. tmp/deploy/sdk/poky-glibc-x86_64-meta-toolchain-cortexa72-poky-linux-toolchain-5.0.sh
```

   Nota: puede que el script se nombre diferente (p.e poky-glibc-x86_64...). Verifique el archivo (o su equivalente) existe al realizar la construcción del toolchain en el directorio (tmp/deploy/sdk)

Prueba de toolchain

Una vez construido e instalado el tolchain, se realizará una prueba simple. Para esto deberá seguir los siguientes pasos:

1. Crear un directorio test_toolchain en /home/usuario/
2. Dentro de directorio test_toolchain, genere un archivo helloworld.c con el siguiente contenido:

```
// 'Hello World!' program

#include <stdio.h>

int main()
{
  printf("Hello World\n");
  return 0;
}
```

3. Ejecutar script para configuración de ambiente de compilación, utilizando el toolchain instalado:

```
. /opt/poky/5.0/environment-setup-cortexa72-poky-linux
```

4. Probar ambiente de compilación: antes de realizar la compilación cruzada, verifique que las variables de ambiente estén configuradas para la compilación. Para esto, ejecute:

```
echo $CC
```

   La salida del comando debería ser algo como:

```
aarch64-poky-linux-gcc -mcpu=cortex-a72 -march=armv8-a+crc+crypto
-fstack-protector-strong  -D_FORTIFY_SOURCE=2 -Wformat -Wformat-security
-Werror=format-security --sysroot=/opt/poky/5.0/sysroots/cortexa72-poky-linux
```

De no ser el caso, debe verificar que el toolchain se ha instalado de la manera correcta. Si es necesario puede volver al paso 3 de la construcción del toolchain.

5. Compilación. Ejecute el siguiente comando para la compilación cruzada de la aplicación *helloworld.c*

```
$CC -o sayhello helloworld.c
```

6. *Ejecución.* Ejecute la aplicación *sayhello*

```
./sayhello
```

¿Por qué no debería verse *'Hello World!'* ? ¿Qué quiere decir la salida de la aplicación?

7. Inserte la uSD con la imagen de Linux creada para Raspberry Pi 4 y copie la aplicación *sayhello* en la raíz del sistema de archivos:

```
sudo cp sayhello /media/rootfs/
```

8. Extraiga la tarjeta SD y conéctela a la Raspberry Pi 4. Encienda la tarjeta, inicie sesión como root e ingrese al directorio raíz del sistema de archivos (dentro de la Raspberry Pi):

```
cd /
```

9. Ejecute finalmente la aplicación *sayhello* dentro de la Raspberry Pi:

```
./sayhello
```

¿Cuál es la salida en este caso? ¿Qué quiere decir?

4. Evaluación

Investigación

1. ¿Qué pasos debe seguir antes de escribir o leer de un puerto de entrada/salida general (GPIO)?
2. ¿Qué comando podría utilizar, bajo Linux, para escribir a un GPIO específico?

Ejercicio práctico:

Descripción

Debe crear una biblioteca dinámica para el manejo de dispositivos de entrada y salida digitales, por medio de gpio, para la plataforma Raspberry Pi 4, en lenguaje C, la cuál ofrecerá las siguientes funciones:

- pinMode (pin, MODE): Permite establecer el modo (entrada/salida) de un pin específico (número de pin/gpio).
- digitalWrite(pin, value): Permite escribir un valor de 0 o 1 en el pin específico configurado como salida.
- digitalRead(pin): Permite leer el estado (0,1) de un pin digital.
- blink(pin, freq, duration): Permite generar un blink (establecer y desestablecer un valor binario) en un pin a una frecuencia determinada, por un tiempo de duración determinado.

Para esta biblioteca, debe utilizar las herramientas generadas con el proyecto Yocto (meta-toolchain) para la construcción de software bajo la metodología de desarrollo cruzado.
Para verificar la biblioteca deberá crear una aplicación que configure dos gpios de la tarjeta como salida y uno como entrada. La aplicación debe escribir valores binarios (0,1) a uno de los gpio configurados como salidas y en el otro establecer un *blink* a una frecuencia determinada por una duración de 5 s. Adicionalmente, se debe leer el valor del pin configurado como entrada y presentarlo en pantalla.

4.1. Entregable

(Subir al tecDigital)

- Archivamiento, con nombre: Estudiante_Eval_Tut_7.tar.gz que incluya los códigos fuente de aplicación y biblioteca, makefiles, scritps, y otros archivos necesarios para la construcción cruzada, siguiente la estructura estándar vista en clase (directorios src, include, lib, bin,). Debe incluir además un archivo README que contenga nombre y carné de los estudiantes, así como la respuesta las preguntas y una guía sobre cómo construir y utilizar su aplicación.